

Nom du projet :

ChatAutoML-Bot : Chatbot LLM pour AutoML sur données tabulaires

1. Contexte et objectifs

ChatAutoML-Bot est une application de type chatbot LLM qui automatise un pipeline complet de Machine Learning (AutoML) pour des données tabulaires supervisées uniquement.

L'objectif est qu'un utilisateur puisse, via un dialogue en langage naturel, charger un dataset tabulaire, laisser le système détecter la tâche (classification ou régression), lancer automatiquement le preprocessing, la recherche de modèles et la sélection du meilleur modèle, puis obtenir une explication claire des résultats.

2. Périmètre fonctionnel

2.1 Types de données pris en charge

- Données tabulaires uniquement (tableau lignes/colonnes, comme Excel ou une table SQL).
- Formats de fichiers autorisés :
 - CSV (.csv) ;
 - Excel (.xlsx, .xls) ;
 - Parquet (.parquet) ;
 - JSON tabulaire (liste de dictionnaires).
- Chaque fichier est chargé dans un `DataFrame` pandas après détection automatique du format.
- Taille maximale configurable (ex. 50–100 Mo) pour rester compatible avec les ressources disponibles.
- Le dataset doit contenir au moins une colonne cible pour l'apprentissage supervisé.

2.2 Limitations explicites

- Pas de données non tabulaires : images, texte brut non structuré, audio, vidéo, séries temporelles complexes ne sont pas supportés dans cette version.
 - Tâches gérées : classification (binaire ou multiclasse) et régression sur données tabulaires uniquement.
-

3. Fonctionnalités du chatbot

- Interface web de type chat (Streamlit ou Gradio) permettant :
 - d'ouvrir une nouvelle session ;
 - d'uploader un ou plusieurs fichiers de données ;
 - d'indiquer ou de confirmer la colonne cible ;
 - de choisir ou d'accepter une métrique principale (classification : accuracy/F1/AUC, régression : MSE/RMSE/R²).
 - Le chatbot doit :
 - analyser automatiquement la colonne cible et détecter le type de tâche (classification ou régression), puis demander confirmation.
 - guider l'utilisateur dans les étapes (upload, cible, lancement de l'AutoML, interprétation des résultats).
 - répondre à des questions sur : les données (déséquilibre, valeurs manquantes), les modèles, les performances, l'overfitting / underfitting et les pistes d'amélioration.
-

4. Pipeline AutoML (backend)

4.1 Chargement et analyse initiale

- Détection de l'extension et lecture du fichier avec la fonction pandas adaptée (`read_csv`, `read_excel`, `read_parquet`, `read_json`).
- Génération d'un résumé automatique :
 - nombre de lignes et de colonnes ;
 - type de chaque feature (numérique, catégorielle, booléenne, éventuellement date) ;
 - pourcentage de valeurs manquantes par colonne ;
 - distribution de la colonne cible (histogramme / comptage).

4.2 Détection du type de tâche

- Analyse de la cible :
 - cible non numérique ou numérique avec peu de valeurs distinctes → classification (binaire ou multiclasse) ;
 - cible numérique continue avec beaucoup de valeurs distinctes → régression.
- Le chatbot annonce le type détecté et demande validation à l'utilisateur.

4.3 Prétraitement générique

- Séparation features (X) / cible (y).
- Gestion des valeurs manquantes :
 - numérique : imputation moyenne ou médiane ;
 - catégoriel : imputation du mode ou d'une catégorie "Unknown".
- Encodage des variables catégorielles (OneHotEncoder ou OrdinalEncoder, via `ColumnTransformer`).
- Normalisation / standardisation des variables numériques pour les modèles sensibles à l'échelle (SVM, KNN, régression linéaire).
- Découpage des données :
 - split train/test ;
 - plus validation croisée k-fold (k paramétrable, par défaut 5) pour évaluer les modèles.

4.4 Gestion des datasets déséquilibrés (classification)

- Analyse de la distribution de la cible pour détecter un déséquilibre (par exemple si une classe > 70–80%).
- Si déséquilibre détecté, application d'une ou plusieurs stratégies :
 - oversampling de la classe minoritaire ;
 - undersampling de la classe majoritaire ;
 - SMOTE ou variante pour générer des exemples synthétiques (via `imblearn.over_sampling.SMOTE`).
- Utilisation prioritaire de métriques robustes au déséquilibre (F1, recall, AUC) dans l'évaluation.
- Le chatbot informe l'utilisateur que le dataset est déséquilibré et indique la stratégie appliquée.

4.5 Recherche de modèles et hyperparamètres (logique AutoML)

Classification – au minimum tester :

- LogisticRegression ;
- RandomForestClassifier ;
- GradientBoostingClassifier ou XGBoost/LightGBMClassifier ;
- SVC ;
- KNeighborsClassifier.

Régression – au minimum tester :

- LinearRegression ;
- RandomForestRegressor ;
- GradientBoostingRegressor ou XGBoost/LightGBMRegressor ;
- KNeighborsRegressor.

Pour chaque modèle :

- définir une petite grille d'hyperparamètres raisonnables (profondeur max, nombre d'arbres, C, k, etc.) ;
- utiliser la validation croisée k-fold pour estimer la performance ;
- enregistrer pour chaque modèle : hyperparamètres testés, score moyen et écart-type sur la métrique principale, scores secondaires.

Optionnel : proposer un mode "AutoML avancé" utilisant auto-sklearn, FLAML ou AutoGluon pour une recherche plus automatisée.

4.6 Gestion overfitting / underfitting

- Comparer systématiquement les performances sur train et validation/test.
- Overfitting détecté si : score train très élevé mais score validation/test nettement plus faible.
- Underfitting détecté si : scores faibles sur train et validation/test.
- Mesures intégrées :
 - usage de modèles régularisés (Ridge/Lasso, LogisticRegression avec pénalisation) ;
 - limitation de la complexité des arbres (profondeur max, min_samples_leaf) ;
 - early stopping pour certains boosters (XGBoost/LightGBM) si utilisé.
- La sélection finale des modèles se fait sur les performances validation/test, pas train, pour éviter de retenir un modèle sur-ajusté.

4.7 Sélection et explication du meilleur modèle

- Génération d'un tableau récapitulatif :
 - modèle ;
 - principaux hyperparamètres ;
 - scores (métrique principale + secondaires, moyenne CV).
 - Sélection automatique et explicite du meilleur modèle selon la métrique principale choisie.
 - Calcul de l'importance des variables pour le meilleur modèle (feature_importances_ ou permutation importance) lorsque c'est possible.
 - Le LLM produit une réponse de type :
 - explication du modèle choisi ;
 - comparaison avec les autres (forces/faiblesses) ;
 - commentaire sur déséquilibre, overfitting/underfitting, limites des données, pistes d'amélioration.
-

5. Architecture technique

- Langage : Python 3.
- Bibliothèques :
 - pandas, numpy (gestion des données) ;
 - scikit-learn (prétraitement, modèles, CV) ;
 - imbalanced-learn (SMOTE, oversampling/undersampling) ;
 - xgboost / lightgbm (optionnel) ;
 - auto-sklearn / FLAML / AutoGluon (optionnel pour AutoML avancé).
- LLM & orchestrateur :
 - API (OpenAI, Mistral, HuggingFace, etc.) ;
 - éventuellement LangChain pour gérer prompts, mémoire de conversation et appels au backend AutoML.
- Interface chatbot :
 - Streamlit (fonctionnalités chat LLM) ou
 - Gradio `ChatInterface`.

Structure de projet proposée :

- `app/` :
 - interface Streamlit/Gradio, endpoints, gestion de session et de l'historique de chat.
 - `automl/` :
 - `data_loader.py` (lecture multi-formats, résumé),
 - `preprocessing.py` (types, imputation, encodage, scaling),
 - `resampling.py` (gestion déséquilibre : SMOTE, over/under-sampling),
 - `model_zoo.py` (définition des modèles et grilles d'hyperparamètres),
 - `search.py` (boucle AutoML + cross-validation),
 - `evaluation.py` (métriques, train/val/test, détection over/underfitting),
 - `selection.py` (classement et choix du meilleur modèle).
 - `llm/` : prompts, formatage des résultats en texte naturel, explications.
 - `datasets/` : jeux de données de démonstration.
 - `config/` : paramètres globaux (métriques, limites de temps, valeurs par défaut).
-

6. Jeux de données de référence à intégrer

Jeux de données tabulaires publics à télécharger et placer dans `datasets/` pour tests et démos :

- Titanic – classification binaire (survie des passagers) – Kaggle :
- <https://www.kaggle.com/c/titanic/data>

-
- Bank Marketing – classification binaire déséquilibrée – UCI :
- <https://archive.ics.uci.edu/dataset/222/bank+marketing>
-
- Iris – classification multiclasse – UCI :
- <https://archive.ics.uci.edu/dataset/53/iris>
-
- Wine Quality – régression ou classification par classes de qualité – UCI :
- <https://archive.ics.uci.edu/dataset/186/wine+quality>
-
- California Housing – régression – description :
- https://www.dcc.fc.up.pt/~ltorgo/Regression/cal_housing.html
- (ou via `sklearn.datasets.fetch_california_housing`).
- Student Performance – régression ou classification – UCI :
- <https://archive.ics.uci.edu/dataset/320/student+performance>
-

Ces éléments constituent le cahier des charges final de ChatAutoML-Bot, centré sur l'AutoML pour données tabulaires supervisées avec interaction via chatbot LLM.